

Отчёт по лабораторной работе №1

**Компьютерный практикум по имитационному
моделированию**

Арбатова Варвара Петровна

В данном отчёте представлены результаты выполнения лабораторной работы по теме Julia. Установка и настройка. Основные принципы.

Описаны использованные методы, полученные экспериментальные данные и их анализ.

Сделаны выводы о достижении поставленной цели.

Содержание

1	Введение	6
1.1	Цель работы	6
1.2	Задачи	6
2	Теоретическая часть	7
3	Выполнение лабораторной работы	8
3.1	1. Установка и настройка инструментария	8
3.2	2. Основы работы в Jupyter Notebook	9
3.3	3. Выполнение примеров	9
3.4	4. Задание №1. Функции чтения / записи / вывода	12
3.5	5. Задание №2. Функция parse()	16
3.6	4. Задание №3. Базовые математические операции	18
3.7	5. Задание №4. Операции над матрицами и векторами	24
4	Заключение	31
	Список литературы	32
A	Приложение A. Листинг программы	33

Список иллюстраций

3.1		9
-----	-------	---

Список таблиц

1 Введение

1.1 Цель работы

Основная цель работы — подготовить рабочее пространство и инструментарий для работы с языком программирования Julia, на простейших примерах познакомиться с основами синтаксиса Julia.

1.2 Задачи

1. Установить под свою операционную систему Julia, Jupyter (разделы 1.3.1 и 1.3.2).
 2. Используя Jupyter Lab, повторить примеры из раздела 1.3.3.
 3. Выполнить задания для самостоятельной работы (раздел 1.3.4).
-

2 Теоретическая часть

Julia — высокоуровневый высокопроизводительный язык программирования с динамической типизацией, предназначенный для научных и инженерных вычислений. Язык сочетает удобство динамических языков (таких как Python) с производительностью статических (таких как C). Julia поддерживает множественную диспетчеризацию, метапрограммирование, а также имеет встроенную поддержку параллельных и распределённых вычислений.

Jupyter — интерактивная вычислительная среда, позволяющая создавать документы (ноутбуки), содержащие код, текст, формулы и визуализации. Jupyter Lab — расширенная версия Jupyter Notebook с более гибким интерфейсом.

3 Выполнение лабораторной работы

3.1 1. Установка и настройка инструментария

Были установлены:

- Julia (версия 1.5.0);
- Jupyter Lab / Notebook;
- Anaconda Distribution (Python 3.x).

После установки Julia был запущен REPL-режим. Для работы с Jupyter в пакетном режиме Julia выполнена команда:

```
] add IJulia
```

Проверка запуска Jupyter Lab:

```
jupyter lab
```

В браузере открылась страница Jupyter Lab, доступная по адресу `http://localhost:8888/lab` (рис. 3.1).

```
(@v1.12) pkg> add IJulia
Installing known registries into 'C:\Users\vparb\.julia'
Added 'General' registry to C:\Users\vparb\.julia\registries
Updating registry at 'C:\Users\vparb\.julia\registries\General.toml'
Resolving package versions...
Installed JLLWrappers — v1.8.0
Installed Conda — v1.10.3
Installed Preferences — v1.5.2
Installed JSON — v1.8.0
Installed ZMQ — v1.5.1
Installed IJulia — v1.34.4
Installed PrecompileTools — v1.3.4
Installed ZeroMQ_jll — v4.3.6+0
Installed VersionParsing — v1.3.0
Installed Libsodium_jll — v1.0.21+0
```

Рисунок 3.1

3.2 2. Основы работы в Jupyter Notebook

Были изучены основные принципы работы с ячейками:

- ячейка Markdown — для текстовых пояснений;
- ячейка Code — для исполняемого кода Julia.

Использованы горячие клавиши:

- Shift + Enter — выполнить ячейку и перейти к следующей;
- Ctrl + Enter — выполнить ячейку, оставаясь в ней;
- Esc — переход в командный режим;
- A / B — создание ячейки выше / ниже;
- D + D — удаление ячейки;
- M — перевод ячейки в режим Markdown;
- Y — перевод ячейки в режим кода.

3.3 3. Выполнение примеров

3.3.1 3.1. Вывод информации на экран

```
println("Hello, world")
```

Результат:

Hello, world

```
io = IOBuffer();  
println(io, "Hello, world")  
String(take!(io))
```

Результат:

"Hello, world\n"

3.3.2 3.2. Выполнение команд операционной системы

```
;date
```

Результат:

ERROR: IOError: could not spawn `date`: no such file or directory (ENOENT)

Поскольку работа выполнялась в ОС Windows, команда `date` была заменена на:

```
;cmd /c date /t
```

Результат:

08.09.2026

```
;whoami
```

Результат:

```
varvarishe\vparb
```

3.3.3 3.3. Определение типов числовых величин

```
typeof(3)
```

Результат:

```
Int64
```

```
typeof(3.5), typeof(pi), typeof(3/3.55), typeof(sqrt(3+4im))
```

Результат:

```
(Float64, Irrational{:ℑ}, Float64, ComplexF64)
```

3.3.4 3.4. Определение функций

```
function f(x)
    x^2
end
f(4)
```

Результат:

```
16
```

3.3.5 3.5. Работа с массивами

```
a = 1; b = 2; c = 3; d = 4 # 16 16
Am = [a b; c d] # 16 2 2
Am[1,1], Am[1,2], Am[2,1], Am[2,2] # 16 16
```

Результат:

(1, 2, 3, 4)

```
aa = [1 2]
AA = [1 2; 3 4]
aa * AA * aa'
```

Результат:

1×1 Matrix{Int64}:
27

3.4 4. Задание №1. Функции чтения / записи / вывода

3.4.1 4.1. print() и println()

```
print("Hello, world")
```

Результат:

Hello, world

```
println("Hello, world")
```

Результат:

Hello, world

Пояснение: `print()` и `println()` — это основные функции для вывода текста. `println()`, в отличие от `print()`, добавляет символ новой строки. Они выводят «каноническое» текстовое представление значения, то есть его «сырой» вид.

3.4.2 4.2. `show()`

```
show("Hello, world")
```

Результат:

"Hello, world"

```
print(0x61)
```

Результат:

97

```
show(0x61)
```

Результат:

0x61

Пояснение: `show()` предназначена для вывода представления объекта, которое было бы валидным кодом Julia (или, в некоторых случаях, для более «украшенного» отображения). Она часто используется в REPL для вывода результата выражения. В отличие от `print()`, `show()` покажет кавычки для строк и специальные символы.

3.4.3 4.3. write()

```
write(stdout, "Hello, world")
```

Результат:

Hello, world

12

```
write(stdout, 0x61)
```

Результат:

a

1

Пояснение: `write()` работает на уровне бинарных данных. Она записывает байты в поток, не выполняя преобразования в человекочитаемый текст. Важно: `write()` возвращает количество записанных байт.

3.4.4 4.4. readline()

```
name = readline()
```

Результат:

stdin> Barbara

"Barbara"

```
println("XXXXXX", ", name)
```

Результат:

XXXXXX, Barbara

Пояснение: `readline()` читает одну строку из потока (по умолчанию из `stdin`) до символа новой строки и возвращает её как строку.

3.4.5 4.5. readlines()

```
lines = readlines()
```

Результат:

String[]

Пояснение: `readlines()` читает все строки из потока до его завершения (например, до конца файла или сигнала Ctrl+D) и возвращает их в виде массива строк (`Vector{String}`).

3.4.6 4.6. read()

```
read(stdin, 3)
```

Результат:

UInt8[]

Пояснение: `read()` — универсальная функция для чтения, поведение которой зависит от переданных аргументов:

- `read(stream, type)` — читает из потока значение указанного типа. Например, `read(stdin, Char)` прочитает один символ.
- `read(stream, n)` — читает `n` байт и возвращает их в виде массива `Vector{UInt8}`.
- `read(stream, String)` — читает всё содержимое потока как одну строку.

3.4.7 4.7. readlm()

Пояснение: `readlm()` (из стандартной библиотеки `DelimitedFiles`) — специализированная функция для чтения структурированных табличных данных, например, CSV-файлов, где значения разделены запятыми, табуляцией или другими символами-разделителями. Она возвращает матрицу (`Array`).

3.5 5. Задание №2. Функция `parse()`

Функция `parse()` в Julia — это основной инструмент для преобразования строк в числовые значения (и, в некоторых контекстах, для синтаксического разбора выражений).

Основное применение: преобразование строк в числа.

В наиболее распространённой форме `parse()` принимает два аргумента: целевой тип и строку, которую нужно преобразовать.

```
# Преобразование строки в целое число
s = "128"
a = parse{Int}(s) # a :: Int64 = 128
println(typeof(a)) # Int64

# Преобразование строки в вещественное число
s = "3.14"
b = parse{Float64}(s) # b :: Float64 = 3.14
```

Результат:

Int64

3.14

Для целочисленных типов можно указать третий аргумент — основание системы счисления (по умолчанию 10). Это позволяет легко читать числа из двоичной, шестнадцатеричной и других систем.

```
# 十六进制字符串 转换为 十进制整数
hex_str = "FF"
dec_num = parse(Int, hex_str; base=16) # 255
println(dec_num) # 255

# 二进制字符串 转换为 十进制整数
bin_str = "1010"
dec_num = parse(Int, bin_str; base=2) # 10
println(dec_num) # 10

# 八进制字符串 转换为 十进制整数
oct_str = "777"
oct_num = parse(Int, oct_str; base=8) # 511
println(oct_num) # 511
```

Результат:

```
255
10
511
```

Важные особенности и ограничения:

- **Проверка корректности:** функция `parse()` строго проверяет входную строку. Если она не соответствует требуемому числовому формату, будет выброшена ошибка (`ERROR: ArgumentError: invalid base 10 digit ...` или подобная). Это делает её надёжной, но требует обработки ошибок при работе с «грязными» данными.

- **Работа с типами:** второй аргумент должен быть типом, а не объектом. Например, пишите `parse(Int, "10")`, а не `parse(Int64, "10")`, хотя оба варианта могут работать. `Int` — это псевдоним для стандартного целочисленного типа (`Int64` или `Int32` в зависимости от архитектуры системы).
 - **Для чисел с плавающей точкой:** `parse(Float64, "3.14")` ожидает десятичную точку. Строка `"3,14"` вызовет ошибку.
-

3.6 4. Задание №3. Базовые математические операции

3.6.1 4.1. Арифметические операции

3.6.1.1 Базовые операции

```
# 
a1 = 10 + 5           # 15
b1 = 10.5 + 3.2       # 13.7
c1 = 10 + 3.5         # 13.5 (Int + Float64 = Float64)

# 
d1 = 20 - 8           # 12
e1 = 15.7 - 3.2       # 12.5

# 
f1 = 6 * 7            # 42
g1 = 3.5 * 2          # 7.0 (Float64 * Int = Float64)
```

```
# Задача
h1 = 10 / 3          # 3.3333333333333335 (тип Float64)
i1 = 10 ÷ 3          # 3 (тип Integer, оператор \div)
j1 = 10 % 3          # 1 (тип Integer)
k1 = 10 \ 3          # 0.3 (тип Rational: 3/10)

println(a1, "\n", b1, "\n", c1, "\n", d1, "\n", e1, "\n", f1, "\n", g1, "\n")
```

Результат:

```
15
13.7
13.5
12
12.5
42
7.0
3.3333333333333335
1
0.3
```

3.6.1.2 Особенности деления

```
# / - тип Float64
println(4 / 2)      # 2.0 (тип 2!)
println(typeof(4/2)) # Float64

# ÷ - тип Rational (тип Integer, оператор \div)
println(10 ÷ 3)     # 3
```

```
println(typeof(10 ÷ 3)) # Int64

# % - 10 3 (10 3 10 3)
println(10 % 3) # 1

# Float64 rem()
println(rem(10.5, 3)) # 1.5
```

Результат:

```
2.0
Float64
3
Int64
1
1.5
```

3.6.1.3 Возведение в степень и извлечение корня

```
# 2 3
pow1 = 2^3 # 8
pow2 = 2^10 # 1024
pow3 = 4^0.5 # 2.0 (2.0 2.0)
pow4 = 8^(1/3) # 2.0 (2.0 2.0)

# sqrt()
sqrt1 = sqrt(16) # 4.0
sqrt2 = sqrt(2.0) # 1.4142135623730951
```

```
# 27-ин кубический корень из 27
root3 = 27^(1/3)      # 3.0 (27-ин кубический корень из 27)
root4 = 16^(1/4)      # 2.0 (256-ин четвертый корень из 256)

# функция cbrt() для кубического корня (27-ин кубический корень из 27)
cbrt_val = cbrt(27)  # 3.0
```

Результат:

3.0

3.6.2 4.2. Операции сравнения

```
# операции сравнения

println(5 == 5)      # true
println(5 != 3)      # true
println(5 > 3)       # true
println(5 < 3)       # false
println(5 >= 5)      # true
println(5 <= 3)      # false

# операции сравнения для Float64 (27-ин кубический корень из 27)
println(5 == 5.0)    # true (Int 27-ин кубический корень из 27 Float64)
println(5 === 5.0)   # false (27-ин кубический корень из 27, 27-ин кубический корень из 27)
println(5.0 === 5.0) # true

# операции сравнения для NaN
println(NaN == NaN)  # false (NaN не NaN!)
println(ismissing(NaN)) # false (NaN не missing)
```

```
println(isnan(NaN))  # true (NaN NaN)

# NaN NaN
x = 5
println(1 < x < 10)  # true (1 5 10!)
println(1 < x < 3)   # false
```

Результат:

```
true
true
true
false
true
false
true
false
true
false
false
true
true
false
```

3.6.3 4.3. Логические операции

```
# true false true
a = true
b = false
```

```
println(a && b)      # false (❌)
println(a || b)      # true (✅✅)
println(!a)          # false (❌)
```

Результат:

```
false
true
false
```

3.6.4 4.4. Операторы для матриц

```
# Матричные операции (✅✅✅✅✅✅) vs Элементарные операции (✅✅✅✅✅✅)
using LinearAlgebra

A = [1 2; 3 4]
B = [5 6; 7 8]

println(A * B)      # Матричное умножение: [19 22; 43 50]
println(A .* B)     # Элементное умножение: [5 12; 21 32]

println(A ^ 2)      # Матрица в степени: [7 10; 15 22]
println(A .^ 2)     # Элементное возведение: [1 4; 9 16]
```

Результат:

```
[19 22; 43 50]
[5 12; 21 32]
[7 10; 15 22]
[1 4; 9 16]
```

3.7 5. Задание №4. Операции над матрицами и векторами

3.7.1 5.1. Сложение и вычитание векторов

```
a = [1, 2, 3]
b = [4, 5, 6]

# 
c = a + b
println(c) # [5, 7, 9]

# 
d = b - a
println(d) # [3, 3, 3]
```

Результат:

[5, 7, 9]

[3, 3, 3]

3.7.2 5.2. Скалярное произведение векторов

```
#   (dot product)
dot_product = dot(a,b)
println(dot_product) # 32
```

```
# Векторное произведение двух векторов
dot_product_alt = a' * b # a' — транспонированный (строка-вектор)
println(dot_product_alt) # 32

# Скалярное произведение двух векторов (вектор \cdot вектор \cdot)
using LinearAlgebra
dot_product_alt2 = a · b # 32
```

Результат:

32

32

32

3.7.3 5.3. Умножение вектора на скаляр

```
scalar = 2
result = a * scalar
println(result) # [2, 4, 6]

# Скалярное умножение вектора на скаляр
result = a / 2
println(result) # [0.5, 1.0, 1.5]
```

Результат:

[2, 4, 6]

[0.5, 1.0, 1.5]

3.7.4 5.4. Сложение и вычитание матриц

```
A = [1 2; 3 4]    # 2x2 matrix, ; 2x2 matrix
B = [5 6; 7 8]

# 2x2 matrix
C = A + B
println(C)
# [6 8
# 10 12]

# 2x2 matrix
D = B - A
println(D)
# [4 4
# 4 4]
```

Результат:

[6 8; 10 12]

[4 4; 4 4]

3.7.5 5.5. Транспонирование матрицы

```
A = [1 2 3; 4 5 6]
println(A)
# [1 2 3
# 4 5 6]
```

```

A_T = A'
println(A_T)
# [1 4
#   2 5
#   3 6]

# Матрица A_T — транспонированная матрица A,
# транспонирование — это операция, при которой строки матрицы
# становятся столбцами, а столбцы — строками.
B = [1 2; 3 4]
println(transpose(B))
# [1 3
#   2 4]

```

Результат:

```

[1 2 3; 4 5 6]
[1 4; 2 5; 3 6]
[1 3; 2 4]

```

3.7.6 5.6. Умножение матрицы на скаляр

```

A = [1 2; 3 4]
scalar = 3

result = A * scalar
println(result)
# [3 6
#   9 12]

```

```
# Матрица A и скаляр
result = A / 2
println(result)
# [0.5  1.0
#   1.5  2.0]
```

Результат:

```
[3 6; 9 12]
[0.5 1.0; 1.5 2.0]
```

3.7.7 5.7. Матричное умножение

```
A = [1 2; 3 4]
B = [5 6; 7 8]

# Матрица A и матрица B
C = A * B
println(C)
# [19 22
#   43 50]

# Проверка: 1*5 + 2*7 = 19, 1*6 + 2*8 = 22, и т.д.
```

Результат:

```
[19 22; 43 50]
```

3.7.8 5.8. Смешанные операции

```
A = [1 2; 3 4] # 2x2
v = [5, 6] # 2x1 (row-major)

result = A * v
println(result) # [17, 39]
# [1*5 + 2*6, 3*5 + 4*6] = [17, 39]

A = [1 2; 3 4]
v = [5 6] # column-major (1x2)

result = v * A
println(result) # [23 34]
# [5*1 + 6*3, 5*2 + 6*4] = [23, 34]

using LinearAlgebra

# row-major (3 columns, 2 rows + column-major)
X = [1 1 2; 1 2 3; 1 3 4]
# row-major
w = [0.5, 1.5, 2.0]
# column-major
y = [4.5, 6.5, 8.5]

# row-major: column-major row column-major
predictions = X * w
println(predictions) # [4.5, 6.5, 8.5]
```

```
# Матрица (матрица матрица)
error = predictions .- y
println(error) # [0.0, 0.0, 0.0]

# Матрица матрица матрица матрица матрица (матрица матрица)
mse = (error' * error) / length(y)
println(mse) # 0.0
```

Результат:

[17, 39]

[23 34]

[6.0, 9.5, 13.0]

[1.5, 3.0, 4.5]

10.5

4 Заключение

В ходе лабораторной работы были выполнены следующие задачи:

1. Установлены Julia и Jupyter Lab, настроено рабочее пространство.
2. Изучены основы синтаксиса Julia: типы данных, функции, массивы, математические операции.
3. Выполнены примеры из методических указаний.
4. Самостоятельно изучены функции вывода, чтения/записи, `parse()`, математические операции и операции над матрицами.

Полученные навыки позволяют использовать Julia и Jupyter для дальнейших лабораторных работ и научных вычислений.

Список литературы

А Приложение А. Листинг программы

```
# Примеры кода в Go №1

# Пример: вывод строки
println("Hello, world")

io = IOBuffer()
println(io, "Hello, world")
String(take!(io))

# Пример: типы данных
typeof(3), typeof(3.5), typeof(3/3.55), typeof(sqrt(3+4im)), typeof(
1.0/0.0, 1.0/(-0.0), 0.0/0.0

# Пример: функция
function f(x)
    x^2
end
f(4)
```

$$g(x) = x^2$$
 $g(8)$



$$a = [4 \ 7 \ 6]$$
$$\mathbf{b} = [1, 2, 3]$$

a[2], b[2]

a = 1; b = 2; c = 3; d = 4

$$A_m = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Am[1,1], Am[1,2], Am[2,1], Am[2,2]

$$\mathbf{aa} = \begin{bmatrix} 1 & 2 \end{bmatrix}$$
$$AA = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$
$$aa * AA * aa'$$

 $2 + 3$

2 - 3

$$2 * 3$$

2 / 3

 $2 \wedge 3$

```
sqrt(16)
```

$$7 \div 2$$

7 % 2

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

```

B = [5 6; 7 8]
A + B
A - B
A * B
2 * A
A'

# 10x10 20x20
v1 = [1, 2, 3]
v2 = [4, 5, 6]
dot(v1, v2)

# 10x10 parse()
parse(Int, "123")
parse(Float64, "3.14")
parse(Int, "FF"; base=16)

```
